一、Phase 1：

1.1 求解程序说明

首先查看 defuser.c 中的函数：

```
static int write_answer(char *out, size_t cap, const uint32_t *values, size_t count)
```

该函数的作用是将 values 数组中的前 count 个整数转换为一行用空格分隔的无符号十进制字符串，并写入 out。其中 out 是存储最终答案字符串的缓冲区地址；cap 是缓冲区的容量，单位为字节；values 是存储计算得到的答案整数；count 是需要输出的整数个数。函数成功时返回 0，未生成答案或输出失败时返回非零值。

1.2 汇编执行及分析：

objdump -d --no-show-raw-insn --disassemble=phase_01 ./bomb > phase_01.S 将第一关 phase_01 的汇编保存到 phase_01.S。

根据 Tutor 教学中的关卡函数接口，以及实际汇编中的参数传递方式，推测 phase_01 的函数形式为：bool phase_01(const char *input, uint64_t challenge); 其中 input 为用户输入的字符串，challenge 为程序生成的 64 位挑战数。

经过一系列基础变化，可知函数入口时：%rdi 存储 input 的地址，%rsi 存储 challenge 的值。

而栈中从下到上依次保存了：%rbp，存储%rsp 的初始状态；%r12，存储 challenge；%rbx，存储 input 地址；同时还预留了 16 字节的局部栈空间。

以及此时%rdi 存储 input 地址，%rsi = 1

结合 Tutor 教学可知，valid_u32_fields(input, 1) 用于检查输入的整数数量、十进制格式及范围，即要求输入一个合法的无符号 32 位整数。

函数返回值存储于%al，test 根据其值更新条件码 ZF。若输入合法，%al=1，ZF=0，jne 跳转至 4020c0；否则执行失败返回代码，返回 0。

其中，失败返回部分的 add、pop 和 ret 用于恢复栈及寄存器并退出函数，xchg %ax,%ax 则是用于代码对齐的空操作。

这部分利用 sscanf 将输入字符串转换为整数。其中 %rdi 为输入字符串地址，%rsi 为格式字符串地址，%rdx 为存储整数的内存地址（%rbp-0x14）。

sscanf 执行后，将解析得到的整数存入栈中，并通过 %eax 返回成功解析的项目数量。随后使用 cmp 和 jne 检查返回值是否为 1，若不是则跳转至失败返回部分。

其中 %r12 保存了原始 challenge，因此首先将其放入 %rdi，作为参数传给 p1_compute。该函数根据 challenge 计算正确答案，并通过 %eax 返回结果。

随后使用 cmp 比较计算结果与此前 sscanf 存储在 -0x14(%rbp) 中的用户输入整数。sete 根据比较结果设置 %al：相等时为 1，否则为 0。因此，可以确定本关需要输入一个无符号整数，且必须满足：input=p1_compute(challenge)

接下来分析 p1_compute 函数

objdump -d --no-show-raw-insn --disassemble=p1_compute ./bomb

该函数接收 challenge 作为参数，首先通过 shr 右移操作提取其中的不同部分，并依次调用 p1_step1、p1_step0 和 p1_step2 计算结果。

其中$A = (uint32_t)$(challenge>>32)

$B = (uint32_t)$(challenge>>16)

$D = (uint32_t)chal$lenge

继续反汇编三个辅助函数

objdump -d --no-show-raw-insn --disassemble=p1_step1 ./bomb

objdump -d --no-show-raw-insn --disassemble=p1_step0 ./bomb

objdump -d --no-show-raw-insn --disassemble=p1_step2 ./bomb

根据汇编中的 lea、xor 和 add 等指令，可以分别得到：

p1_step1：$R_1 = (B \oplus 0xa98d) + 4A$　　p1_step0：$R_0 = (A + 0x1283) \oplus D$

p1_step2：$R_2 = (5R_0 + R_1) \oplus 0x622f$　　因此，p1_compute 最终返回：R_2

测试 seed 20261028

```
fanmo@LAPTOP-R7Q6LOMR:~/ICS/ics-2026-bomblab$ ./defuser --seed 20261028 --phase 1 | ./bomb --seed 20261028 --phase 1
第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

Phase 1 challenge: 0xf86c17686e390f05
Input (unsigned decimal integers):
Phase 1 defused.

[1 已解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

二、Phase 2：

objdump -d --no-show-raw-insn --disassemble=phase_02 ./bomb > phase_02.S

根据函数入口的汇编可知，%r12 保存输入字符串地址，%r13 保存原始 challenge。随后将 %esi 设置为 6，调用 valid_u32_fields(input, 6)，检查输入是否为 6 个合法的无符号 32 位整数。

随后调用 sscanf，通过 %rdi 传递输入字符串地址，%rsi 传递格式字符串地址，其余参数通过寄存器及栈传递。解析得到的 6 个整数依次存储在 -0x40(%rbp)至-0x2c(%rbp)的位置，每个整数占 4 字节。随后检查%eax 是否为 6，若不是则返回失败。

接下来先将 challenge 复制到 %rax，右移 29 位，再与 0xf 按位与，提取右移结果的低 4 位。随后与第一个输入整数进行比较，若不相等则返回失败。因此，第一个整数应满足：$values[0] = (challenge \gg 29)\ \&\ 0xf$

随后程序进入循环：其中 %rdx 作为循环索引，从 1 开始，每次增加 1，依次检查剩余的 5 个整数。

具体，lea (%rdx，%rdx，1)，%ecx 计算两倍索引，随后 shr %cl，%rax 将 challenge 右移相应位数，再通过 and $0xf，%eax 提取低 4 位，并加上 0xe。同时，%esi 最初保存前一个输入整数，lea (%rsi，%rsi，2)，%ecx 计算其 3 倍，再通过 sub %edx，%ecx 减去循环索引。最后将两部分结果按位异或，并与当前输入整数比较。

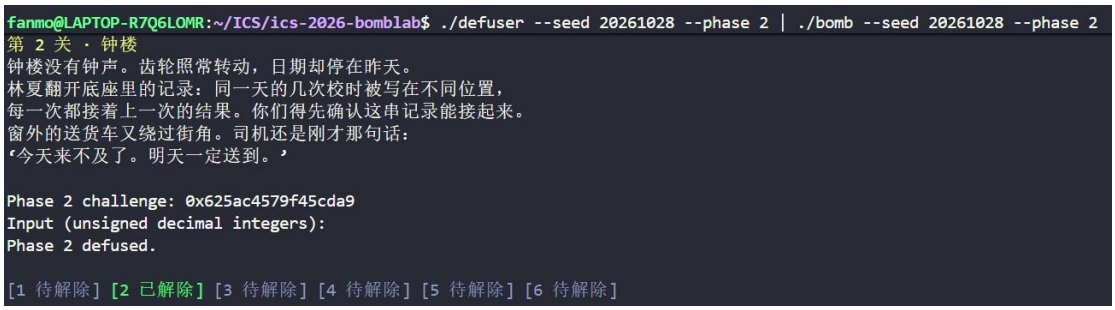
因此，可以得到(其中 $i = 1,2,3,4,5$)：

$$values[i] = \big(((challenge \gg 2i)\ \&\ 0xf) + 14\big) \oplus (3values[i-1] - i)$$

当循环完成且所有比较均相等时，程序保留成功返回值并退出函数；若任意一个整数不满足条件，则跳转至失败分支，返回 0。

综上，第二关要求输入 6 个无符号 32 位整数。第一个整数由 challenge 直接计算，其余 5 个整数通过上述递推关系依次得到，据此即可完成 solve_phase_2() 的求解。

测试 seed 20261028



三、Phase 3：

objdump -d --no-show-raw-insn --disassemble=phase_03 ./bomb > phase_03.S

根据函数入口汇编可知，%rbx 保存输入字符串地址，%r12 保存原始 challenge。随后将 %esi 设置为 2，调用 valid_u32_fields(input, 2)，检查输入是否为 2 个合法的无符号 32 位整数。

随后通过 sscanf 读取两个整数，分别存储在 -0x18(%rbp) 和 -0x14(%rbp) 中，并检查成功读取的整数数量是否为 2。

接下来，程序将 challenge 的低 32 位放入 %edi，并与 0x7 按位与，提取低 3 位，再与第一个输入整数比较。因此，第一个整数必须满足：

$$values[0] = challenge \,\&\, 0x7$$

随后程序将 challenge 分别右移 3 位和 32 位，并将结果的低 32 位按位异或，作为第二个参数传入 p3_dispatch；第一个参数为此前计算的 challenge 低 3 位。

继续反汇编该函数：

objdump -d --no-show-raw-insn --disassemble=p3_dispatch ./bomb

其中 cmp 和 ja 用于检查分支索引是否超过 7，随后通过 jmp *0x405820 (,%rax,8) 根据索引从跳转表中读取目标地址，选择不同的计算分支。

使用 GDB 查看跳转表：gdb -batch -ex 'x/8gx 0x405820' ./bomb

结合跳转表地址和各分支的汇编，可知索引 0～7 实际对应的函数依次为：

p3_case_2、p3_case_0、p3_case_6、p3_case_4、

p3_case_3、p3_case_5、p3_case_1、p3_case_7。

继续反汇编 p3_case_0 至 p3_case_7，得到各分支的计算公式。

令：$x = (uint32_t)(challenge \gg 32) \oplus (uint32_t)(challenge \gg 3)$

则第二个整数的计算结果为：

0→$(x \oplus 0x408d) + (x \gg 1)$

1→$(2x) \oplus 0x2c7f$

2→$(x \oplus 0xa7c2) + (x \gg 1)$

3→(5x+0x14f91)

4→(5x+0x735)

5→(3x+0xb2b4)

6→(5x+0x44ad9)

7→((4x)\oplus0x448c)

最后，程序将 p3_dispatch 的返回值与第二个输入整数比较，通过 sete 设置最终返回值。只有两个整数均满足对应条件时，第三关才会通过。

综上，第三关需要输入两个无符号 32 位整数。第一个整数由 challenge 的低 3 位确定，第二个整数则根据跳转表选择对应分支计算得到，据此即可完成 solve_phase_3() 的求解。

测试 seed：20261028

```
fanmo@LAPTOP-R7Q6LOMR:~/ICS/ics-2026-bomblab$ ./defuser --seed 20261028 --phase 3 | ./bomb --seed 20261028 --phase 3
第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0x1330a455fa5a79a6
Input (unsigned decimal integers):
Phase 3 defused.

[1 待解除] [2 待解除] [3 已解除] [4 待解除] [5 待解除] [6 待解除]
```

四、Phase 4：

objdump -d --no-show-raw-insn --disassemble=phase_04 ./bomb > phase_04.S

根据函数入口汇编可知，%rbx 保存输入字符串地址，%r12 保存原始 challenge。随后将 %esi 设置为 1，调用 valid_u32_fields(input, 1)，检查输入是否为一个合法的无符号 32 位整数。

随后通过 sscanf 读取该整数，将其存储在 -0x14(%rbp) 中，并检查是否成功读取一个整数，否则返回失败。

接下来，程序通过 shr、and 和 xor 等指令处理 challenge，并调用 p4_walk。该函数接收四个参数，分别为：

edi = ((uint32_t)challenge) & 0x7f，esi = 0，edx = 127

$ecx = (uint32_t)(challenge \gg 7) \oplus (uint32_t)(challenge \gg 32)$

继续反汇编 p4_walk：

objdump -d --no-show-raw-insn --disassemble=p4_walk ./bomb

根据汇编可知，该函数采用递归二分查找的方式进行计算。其中第一个参数为目标值，第二、三个参数分别为搜索区间的下界和上界，第四个参数参与返回值的计算。

程序首先通过 sub、shr 和 add 计算区间中点：

$$mid = low + \frac{high - low}{2}$$

随后通过 cmp 比较目标值与中点，根据比较结果分为三种情况：

若 target=mid，直接返回 $(mid + 250) \oplus salt$。

若 target<mid，递归搜索左半区间$[low, mid - 1]$。

若 target>mid，递归搜索右半区间 $[mid + 1, high]$。

对于后两种情况，程序在递归返回后，先将子函数的返回值乘以 5，再与 mid+salt 按位异或。其中左分支最后还需异或 0x3，右分支则异或 0xd。

因此，令 (F(target,low,high,salt)) 表示 p4_walk 的返回值，其递归关系为：

$$F = \begin{cases} (mid + 250) \oplus salt, & target = mid \\ (5F_L \oplus (mid + salt)) \oplus 0x3, & target < mid \\ (5F_R \oplus (mid + salt)) \oplus 0xd, & target > mid \end{cases}$$

最后，phase_04 将 p4_walk 的返回值与此前 sscanf 读取的整数进行比较，通过 sete 设置最终返回值，相等时通过第四关，否则返回失败。

测试 seed：20261028

```
fanmo@LAPTOP-R7Q6LOMR:~/ICS/ics-2026-bomblab$ ./defuser --seed 20261028 --phase 4 | ./bomb --seed 20261028 --phase 4
第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：‘这次别凭房间号猜，跟着记录走。’

Phase 4 challenge: 0x33bef1221bd56983
Input (unsigned decimal integers):
Phase 4 defused.

[1 待解除] [2 待解除] [3 待解除] [4 已解除] [5 待解除] [6 待解除]
```

五、Phase 5：

objdump -d --no-show-raw-insn --disassemble=phase_05 ./bomb > phase_05.S

根据函数入口汇编可知，%rbx 保存输入字符串地址，%r12 保存原始 challenge。随后将 %esi 设置为 2，调用 valid_u32_fields(input, 2)，检查输入是否为 2 个合法的无符号 32 位整数。

随后通过 sscanf 读取两个整数，分别存储在 -0x1c(%rbp) 和 -0x18(%rbp) 中，并检查是否成功读取两个整数。接下来，程序将 challenge 放入 %rdi，将局部变量 -0x14(%rbp) 的地址放入 %rsi，调用 p5_traverse。该函数通过 %eax 返回一个计算结果，同时通过指针参数写入另一个整数。

继续反汇编该函数：

objdump -d --no-show-raw-insn --disassemble=p5_traverse ./bomb

根据汇编中的 and $0x7,%eax 和 mov 0x405b60(,%rax,8),%rdx 可知，程序利用 challenge 的低 3 位作为索引，从 0x405b60 开始的指针表中选择起始链表节点。使用 GDB 查看指针表及节点数据：

gdb -batch -ex 'x/8gx 0x405b60' ./bomb

gdb -batch -ex 'x/24gx 0x405ba0' ./bomb

根据内存数据可知，链表共有 12 个节点，每个节点占 16 字节，依次存储下一个节点的地址、节点整数值以及参与异或运算的整数。8 个起始节点对应的节点编号依次为：2、6、8、3、5、11、4、1。

随后，程序通过 shr、xor、add 等指令计算初始结果，并根据 challenge 决定链表遍历次数。

其中，起始节点索引和额外遍历次数分别为：

$$index = challenge \,\&\, 0x7，\ steps = (challenge \gg 3)0x3$$

程序首先处理起始节点，计算：

$$result=((uint32_t)(challenge\gg 6)\oplus(uint32_t)(challenge\gg 32)\oplus tag\oplus 0x102b)+data$$

其中 tag 和 data 分别表示当前节点的异或值和整数值。

随后沿链表额外移动 steps 次，每次读取下一节点，并更新累计结果：

$$result = (result \oplus tag) + data$$

同时将当前节点的整数值写入指针参数所指向的内存。因此，遍历结束后，p5_traverse 返回累计计算结果，并通过指针保存最后访问节点的整数值。

最后，phase_05 将第一个输入整数与函数返回值比较，将第二个输入整数与指针保存的结果比较。只有两者均相等时，程序返回成功。

综上，第五关要求输入两个无符号 32 位整数。第一个整数为链表遍历的累计计算结果，第二个整数为最后访问节点的整数值，据此即可完成 solve_phase_5() 的求解。

测试 seed：20261028

```
fanmo@LAPTOP-R7Q6LOMR:~/ICS/ics-2026-bomblab$ ./defuser --seed 20261028 --phase 5 | ./bomb --seed 20261028 --phase 5
第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0xfa34cb32b3ae97c7
Input (unsigned decimal integers):
Phase 5 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 已解除] [6 待解除]
```

六、Phase 6：

objdump -d --no-show-raw-insn --disassemble=phase_06 ./bomb > phase_06.S

根据函数入口汇编可知，%rbx 保存输入字符串地址，%r12 保存原始 challenge。随后将 %esi 设置为 1，调用 valid_u32_fields(input, 1)，检查输入是否为一个合法的无符号 32 位整数。

随后通过 sscanf 读取整数，存储在 -0x14(%rbp) 中，并检查是否成功读取一个整数。接下来，程序将 challenge 传给 p6_dispatch。继续反汇编：

objdump -d --no-show-raw-insn --disassemble=p6_dispatch ./bomb

根据汇编可知，程序首先通过 shr 和 xor 处理 challenge，得到第一次函数调用的参数：

$$x=(uint32_t)(challenge\gg 6)\oplus(uint32_t)(challenge\gg 32)$$

随后通过 and 提取 challenge 的不同位，分别确定两次函数调用的索引：

$$i = challenge0x7，\ j = (challenge \gg 3)0x7$$

其中 call *0x405e00(,%rax,8) 和 call *0x405e00(%rbx) 表明程序通过函数指针表调用对应的计算函数。使用 GDB 查看函数指针表：

gdb -batch -ex 'x/8gx 0x405e00' ./bomb

结合指针表地址可知，索引 0～7 对应的函数依次为：

p6_op_4、p6_op_2、p6_op_1、p6_op_5、
p6_op_3、p6_op_0、p6_op_0、p6_op_3。

继续反汇编 p6_op_0 至 p6_op_5，得到：

p6_op_0　$((x+0xecdc)\oplus(x\gg 2))$

p6_op_1　$(ROL_{32}(x,3)\oplus 0x8f36)$

p6_op_2　$((4x+0x98fc)\oplus 7x)$

p6_op_3　　$(ROR_{32}(x,2))$
p6_op_4　　$((x \oplus 0xf9b1)+(x \ll 5))$
p6_op_5　　$(((x \gg 5) \oplus x)+0xac08)$

其中，ROL 和 ROR 分别表示循环左移和循环右移，因此，p6_dispatch 的整体计算过程为：

$$\text{result}_1 = \text{p6_ops}[i](x)，\ \text{result}_2 = \text{p6_ops}[j](\text{result}_1 \oplus 0x3926)$$

$$\text{answer} = \text{result}_2 + 0x88ef$$

程序首先根据索引 (i) 调用第一个函数，将返回值与 0x3926 按位异或，再根据索引 (j) 调用第二个函数，最后将结果加上 0x88ef。

随后，phase_06 将 p6_dispatch 的返回值与用户输入整数进行比较，通过 sete 设置最终返回值，相等时通过第六关，否则返回失败。

综上，第六关要求输入一个无符号 32 位整数。该整数由 challenge 决定两次函数调用的索引和初始参数，再经过两次计算得到，据此即可完成 solve_phase_6() 的求解。

测试 seed：20261028

```
fanmo@LAPTOP-R7Q6LOMR:~/ICS/ics-2026-bomblab$ ./defuser --seed 20261028 --phase 6 | ./bomb --seed 20261028 --phase 6
第 6 关 · 交接室
交接室的灯还亮着。桌上留着一份没有签完的记录。
你们刚才找回的时间、路线、存档和设备状态都到了这里。
林夏把记录摊开，逐项核对。你接上最后一台终端。
它会根据当前状态选择处理例程，再检查最后的交接结果。
走廊里传来送货车的声音。你们把门掩上，继续看屏幕。

Phase 6 challenge: 0x8a392b607a4bb6cc
Input (unsigned decimal integers):
Phase 6 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 已解除]
```

七、Honor：

objdump -d --no-show-raw-insn --disassemble=secret_gate ./bomb > secret_gate.S
objdump -d --no-show-raw-insn --disassemble=secret_copy ./bomb > secret_copy.S
objdump -d --no-show-raw-insn --disassemble=secret_phase ./bomb > secret_phase.S
objdump -d --no-show-raw-insn --disassemble=secret_mix ./bomb > secret_mix.S

首先分析 secret_gate。该函数通过 strlen 检查输入字符串长度，要求长度在 36～163 字节之间，随后调用 secret_copy 将输入字符串复制到栈中。根据 secret_copy 的汇编可知，该函数逐字节复制字符串，直到遇到 \0 结束符。

在 secret_gate 中，复制目标地址为 -0xd0(%rbp)，而 cmpl $0x4e45504f, -0xb0(%rbp) 检查的是偏移 32 字节处的内容。由于采用小端序，该值对应字符串 OPEN。随后程序从偏移 36 字节处调用 secret_phase，检查后续输入。

因此，Honor 的输入格式为：前 32 个填充字符、OPEN，以及后续三个无符号整数。接下来分析 secret_phase。该函数首先调用 valid_u32_fields(input, 3)，检查后续输入是否为三个合法的无符号 32 位整数，再通过 sscanf 读取并分别比较。其中第一个整数由 context 的低 2 位确定：

$$v_0 = \text{context} \,\&\, 0x3$$

随后程序通过 shr 和 xor 处理 challenge 与 context，令

$$a = (\text{uint32_t})(\text{challenge} \gg 32) \oplus (\text{uint32_t})\text{challenge}$$

$$b = (uint32_t)(context \gg 32) \oplus (uint32_t)context$$

继续反汇编 secret_mix：

objdump -d --no-show-raw-insn --disassemble=secret_mix ./bomb

根据其中的 lea 和 xor 指令，可以得到：

$$\text{secret_mix}(x) = (5x + 0xf87b) \oplus 0x4c7e$$

结合 secret_phase 的汇编，第二个整数应满足：

$$v_1 = \text{secret_mix}(a \oplus b) \oplus (0xf87b\, v_0 \oplus (\text{uint32_t})(\text{context} \gg 16))$$

第三个整数则满足：

$$v_2 = v_1 \oplus (0x4c7e + b + v_0 + (a \gg 3))$$

最后，在 solve_secret() 中根据 challenge 和 context 计算三个整数，并使用 snprintf 将 32 个填充字符、OPEN 和三个整数组合成完整的答案字符串。

综上，Honor 与前六关不同，不仅需要根据两个 64 位参数计算三个整数，还需要满足指定的字符串长度及固定偏移位置的字符检查。根据上述关系即可完成 solve_secret() 的求解。

八、工具使用说明

本实验主要参考课程提供的 BombLab 实验说明、Tutor 教学资料，以及《深入理解计算机系统》（CSAPP）中有关 x86-64 汇编和程序控制的知识。实验过程中，使用 objdump 对可执行文件进行反汇编，使用 GDB 查看内存数据、跳转表及函数指针表，并通过 make 和课程提供的测试脚本完成编译与验证。此外，主要使用 ChatGPT 辅助理解汇编指令、梳理函数调用关系、推导计算逻辑及整理实验报告。实验报告记录了各关卡的主要分析依据及求解过程，未将 AI 生成的内容直接作为未经验证的个人分析成果。